

Liste, tuple, dizionari, stringhe, set e array in Python

Questa guida raccoglie le strutture dati Python più usate nello studio iniziale e intermedio, con una mini tabella introduttiva e poi tabelle separate delle funzioni o dei metodi più importanti. La documentazione ufficiale dei tipi built-in descrive list, tuple, dict, set e str, mentre gli array del modulo array sono documentati come struttura compatta per valori di base tipizzati.

Panoramica rapida

Struttura	Ordinata	Modificabile	Duplicati	Indicizzazione	Nota rapida
list	Sì	Sì	Sì	Sì	Sequenza molto flessibile, può contenere tipi misti.
tuple	Sì	No	Sì	Sì	Sequenza immutabile, utile per dati fissi.
dict	Sì, mantiene l'ordine di inserimento	Sì	Chiavi uniche	Per chiave, non per posizione	Mappa chiavi hashable a valori arbitrari.
str	Sì	No	Sì	Sì	Sequenza immutabile di caratteri Unicode.
set	No	Sì	No	No	Collezione non ordinata di elementi distinti hashable.
array.array	Sì	Sì	Sì	Sì	Simile a una lista ma con elementi tutti dello stesso tipo.

Regole utili da ricordare

Le strutture list, dict, set e array.array sono modificabili, quindi puoi aggiungere, rimuovere o cambiare elementi dopo la creazione. Le strutture tuple e str sono immutabili, quindi il contenuto non può essere cambiato in place e ogni operazione di modifica produce di fatto un nuovo oggetto.

Le liste, tuple, stringhe e array supportano indicizzazione e slicing perché sono sequenze. I set non supportano indexing perché sono collezioni non ordinate, mentre i dizionari si accedono tramite chiavi e non tramite posizione.

Le liste sono tra le strutture più usate in Python e il tutorial ufficiale dedica una sezione specifica ai loro metodi principali. Sono adatte quando serve una sequenza modificabile e ordinata.

Liste

Metodo o funzione	Sintassi	Esempio	Come si usa
list()	list(iterable)	list('abc')	Crea una lista da un iterabile.

Metodo o funzione	Sintassi	Esempio	Come si usa
append	<code>lst.append(x)</code>	<code>nums.append(10)</code>	Aggiunge un elemento in fondo alla lista.
extend	<code>lst.extend(iterable)</code>	<code>nums.extend([4, 5])</code>	Aggiunge più elementi da un iterabile.
insert	<code>lst.insert(i, x)</code>	<code>nums.insert(1, 99)</code>	Inserisce un elemento in una posizione specifica.
remove	<code>lst.remove(x)</code>	<code>nums.remove(99)</code>	Rimuove la prima occorrenza di un valore.
pop	<code>lst.pop([i])</code>	<code>nums.pop()</code>	Rimuove e restituisce un elemento, di default l'ultimo.
clear	<code>lst.clear()</code>	<code>nums.clear()</code>	Svuota completamente la lista.
index	<code>lst.index(x[, start[, end]])</code>	<code>nums.index(5)</code>	Restituisce la posizione della prima occorrenza.
count	<code>lst.count(x)</code>	<code>nums.count(2)</code>	Conta quante volte compare un valore.
sort	<code>lst.sort(key=None, reverse=False)</code>	<code>nums.sort()</code>	Ordina la lista in place.
reverse	<code>lst.reverse()</code>	<code>nums.reverse()</code>	Inverte l'ordine degli elementi in place.
len	<code>len(lst)</code>	<code>len(nums)</code>	Restituisce il numero di elementi.
sorted	<code>sorted(iterable, key=None, reverse=False)</code>	<code>sorted(nums)</code>	Restituisce una nuova lista ordinata senza modificare l'originale.

Tuple

Le tuple sono sequenze immutabili e vengono usate quando i dati non devono cambiare oppure per rappresentare gruppi fissi di valori. Non hanno molti metodi proprio perché non sono modificabili.

Metodo o funzione	Sintassi	Esempio	Come si usa
<code>tuple()</code>	<code>tuple(iterable)</code>	<code>tuple([1, 2, 3])</code>	Crea una tupla da un iterabile.
count	<code>tpl.count(x)</code>	<code>t.count(2)</code>	Conta quante volte compare un valore.
index	<code>tpl.index(x[, start[, end]])</code>	<code>t.index(3)</code>	Restituisce la posizione della prima occorrenza.
len	<code>len(tpl)</code>	<code>len(t)</code>	Restituisce il numero di elementi.
slicing	<code>tpl[start:stop:step]</code>	<code>t[1:3]</code>	Estrae una sotto-tupla senza modificare l'originale.

Dizionari

I dizionari sono la struttura mapping standard di Python: associano chiavi hashable a valori arbitrari. Sono molto usati quando serve accedere rapidamente a un valore tramite nome o chiave.

Metodo o funzione	Sintassi	Esempio	Come si usa
<code>dict()</code>	<code>dict(**kwargs)</code> o <code>dict(mapping)</code>	<code>dict(nome='Anna', eta=23)</code>	Crea un dizionario.
accesso per chiave	<code>d[key]</code>	<code>d['nome']</code>	Restituisce il valore associato a una chiave.
<code>get</code>	<code>d.get(key, default=None)</code>	<code>d.get('eta', 0)</code>	Recupera un valore senza errore se la chiave manca.
<code>keys</code>	<code>d.keys()</code>	<code>d.keys()</code>	Restituisce una vista delle chiavi.
<code>values</code>	<code>d.values()</code>	<code>d.values()</code>	Restituisce una vista dei valori.
<code>items</code>	<code>d.items()</code>	<code>d.items()</code>	Restituisce coppie chiave-valore.
<code>update</code>	<code>d.update(other)</code>	<code>d.update({'citta': 'Roma'})</code>	Aggiunge o aggiorna coppie chiave-valore.
<code>pop</code>	<code>d.pop(key[, default])</code>	<code>d.pop('eta')</code>	Rimuove una chiave e restituisce il valore.
<code>popitem</code>	<code>d.popitem()</code>	<code>d.popitem()</code>	Rimuove e restituisce l'ultima coppia inserita.
<code>clear</code>	<code>d.clear()</code>	<code>d.clear()</code>	Svuota il dizionario.
<code>len</code>	<code>len(d)</code>	<code>len(d)</code>	Restituisce il numero di coppie chiave-valore.

Stringhe

Le stringhe sono sequenze immutabili di caratteri Unicode e implementano molte operazioni di manipolazione testuale. Sono fondamentali per parsing, pulizia dati e gestione testo.

Metodo o funzione	Sintassi	Esempio	Come si usa
<code>str()</code>	<code>str(object='')</code>	<code>str(123)</code>	Converte un oggetto in stringa.
<code>lower</code>	<code>s.lower()</code>	<code>'CIAO'.lower()</code>	Converte in minuscolo.
<code>upper</code>	<code>s.upper()</code>	<code>'ciao'.upper()</code>	Converte in maiuscolo.
<code>strip</code>	<code>s.strip(chars=None)</code>	<code>' ciao '.strip()</code>	Rimuove spazi o caratteri ai bordi.

replace	s.replace(old, new[, count])	'ciao mondo'.replace('mondo', 'Giovanni')	Sostituisce parti del testo.
---------	---------------------------------	---	------------------------------

Metodo o funzione	Sintassi	Esempio	Come si usa
split	<code>s.split(sep=None, maxsplit=-1)</code>	<code>'a,b,c'.split(',')</code>	Divide una stringa in lista.
join	<code>sep.join(iterable)</code>	<code>'-'.join(['a','b'])</code>	Unisce più stringhe con un separatore.
find	<code>s.find(sub[, start[, end]])</code>	<code>'python'.find('th')</code>	Restituisce la posizione di una sottostringa o -1.
startswith	<code>s.startswith(prefix)</code>	<code>'python'.startswith('py')</code>	Controlla se la stringa inizia con un prefisso.
endswith	<code>s.endswith(suffix)</code>	<code>'report.csv'.endswith('.csv')</code>	Controlla se la stringa finisce con un suffisso.
len	<code>len(s)</code>	<code>len('ciao')</code>	Restituisce il numero di caratteri.

Set

I set sono collezioni di elementi distinti e hashable, molto utili per membership testing, rimozione duplicati e operazioni matematiche tra insiemi. Non supportano indicizzazione né slicing.

Metodo o funzione	Sintassi	Esempio	Come si usa
set()	<code>set(iterable)</code>	<code>set([1, 2, 2, 3])</code>	Crea un set e rimuove automaticamente i duplicati.
add	<code>s.add(elem)</code>	<code>s.add(10)</code>	Aggiunge un elemento.
update	<code>s.update(*others)</code>	<code>s.update([4,5])</code>	Aggiunge più elementi da iterabili.
remove	<code>s.remove(elem)</code>	<code>s.remove(3)</code>	Rimuove un elemento, con errore se manca.
discard	<code>s.discard(elem)</code>	<code>s.discard(99)</code>	Rimuove un elemento senza errore se non esiste.
pop	<code>s.pop()</code>	<code>s.pop()</code>	Rimuove e restituisce un elemento arbitrario.
clear	<code>s.clear()</code>	<code>s.clear()</code>	Svuota il set.

Metodo o funzione	Sintassi	Esempio	Come si usa
union	s.union(*others)	a.union(b)	Unisce gli elementi di più set.
intersection	s.intersection(*others)	a.intersection(b)	Restituisce gli elementi comuni.
difference	s.difference(*others)	a.difference(b)	Restituisce gli elementi presenti solo nel primo set.
symmetric_difference	s.symmetric_difference(other)	a.symmetric_difference(b)	Restituisce gli elementi presenti in un solo set dei due.
len	len(s)	len(s)	Restituisce il numero di elementi distinti.

Array

Il modulo array definisce un tipo di array compatto per valori di base come interi e float, con elementi vincolati da un typecode. Si comporta in modo simile a una lista, ma richiede elementi dello stesso tipo e usa una rappresentazione più compatta.

Metodo o funzione	Sintassi	Esempio	Come si usa
array.array	array.array(typecode[, initializer])	array('i', [1, 2, 3])	Crea un array tipizzato; ad esempio 'i' indica interi.
append	arr.append(x)	arr.append(4)	Aggiunge un elemento alla fine.
extend	arr.extend(iterable)	arr.extend([5, 6])	Aggiunge più elementi.
insert	arr.insert(i, x)	arr.insert(1, 99)	Inserisce un elemento in una posizione.
remove	arr.remove(x)	arr.remove(99)	Rimuove la prima occorrenza di un valore.
pop	arr.pop([i])	arr.pop()	Rimuove e restituisce un elemento.
count	arr.count(x)	arr.count(2)	Conta quante volte compare un valore.
index	arr.index(x)	arr.index(3)	Restituisce la posizione della prima occorrenza.
reverse	arr.reverse()	arr.reverse()	Inverte l'ordine in place.
tolist	arr.tolist()	arr.tolist()	Converte l'array in lista Python.

Metodo o funzione	Sintassi	Esempio	Come si usa
fromlist	<code>arr.fromlist(list)</code>	<code>arr.fromlist([7,8])</code>	Aggiunge elementi prendendoli da una lista.
len	<code>len(arr)</code>	<code>len(arr)</code>	Restituisce il numero di elementi.

Quando usare cosa

Le liste sono la scelta più generica quando serve una sequenza modificabile di elementi anche di tipi diversi. Le tuple sono utili quando i dati devono restare fissi, i dizionari quando lavori con coppie chiave-valore, i set quando servono unicità e operazioni insiemistiche, le stringhe per il testo e gli array del modulo array quando vuoi una struttura compatta di valori numerici tutti dello stesso tipo.